

Progettazione Database

Introduzione:

Si vuole progettare una base di dati per l'organizzazione e la gestione di un campionato di pallacanestro di livello regionale.

L'obiettivo del progetto è di tenere traccia delle informazioni anagrafiche degli enti e delle persone coinvolte in questa competizione e di monitorare l'andamento del campionato, incluse le prestazioni dei singoli giocatori.

Requisiti:

Si terrà conto di diverse società sportive, di cui si vuole memorizzare il codice di affiliazione, il nome ufficiale, l'indirizzo della sede e dei contatti vari (e-mail e telefono).

Le varie società potranno iscrivere una o più squadre al campionato, le cui caratteristiche di interesse sono l'ID univoco, il nome commerciale (comprendendo eventuali sponsor) e i colori (delle divise di casa e trasferta).

Inoltre, ad ogni squadra è associato un palazzetto, quello in cui si disputano le partite in casa. Ci interesserà prendere nota di alcuni attributi relativi al palazzetto, quali un ID, il nome, l'indirizzo, la città e la capienza.

Ogni squadra è formata dai membri "tesserati", ovvero i giocatori e gli allenatori. Per tutti i tesserati ci interessa di tenere conto di diversi campi: Codice Fiscale, nome, cognome e data di nascita. Per gli allenatori verrà memorizzato anche il livello del proprio tesserino (Giovanili, Dilettanti, Professionisti). Esclusivamente per i giocatori, invece, si terrà conto anche dell'altezza, il ruolo, il numero di maglia e la data di scadenza del certificato medico. Sarà inoltre utile tenere conto dell'anno in cui un giocatore ha iniziato a militare per la squadra in cui è attualmente.

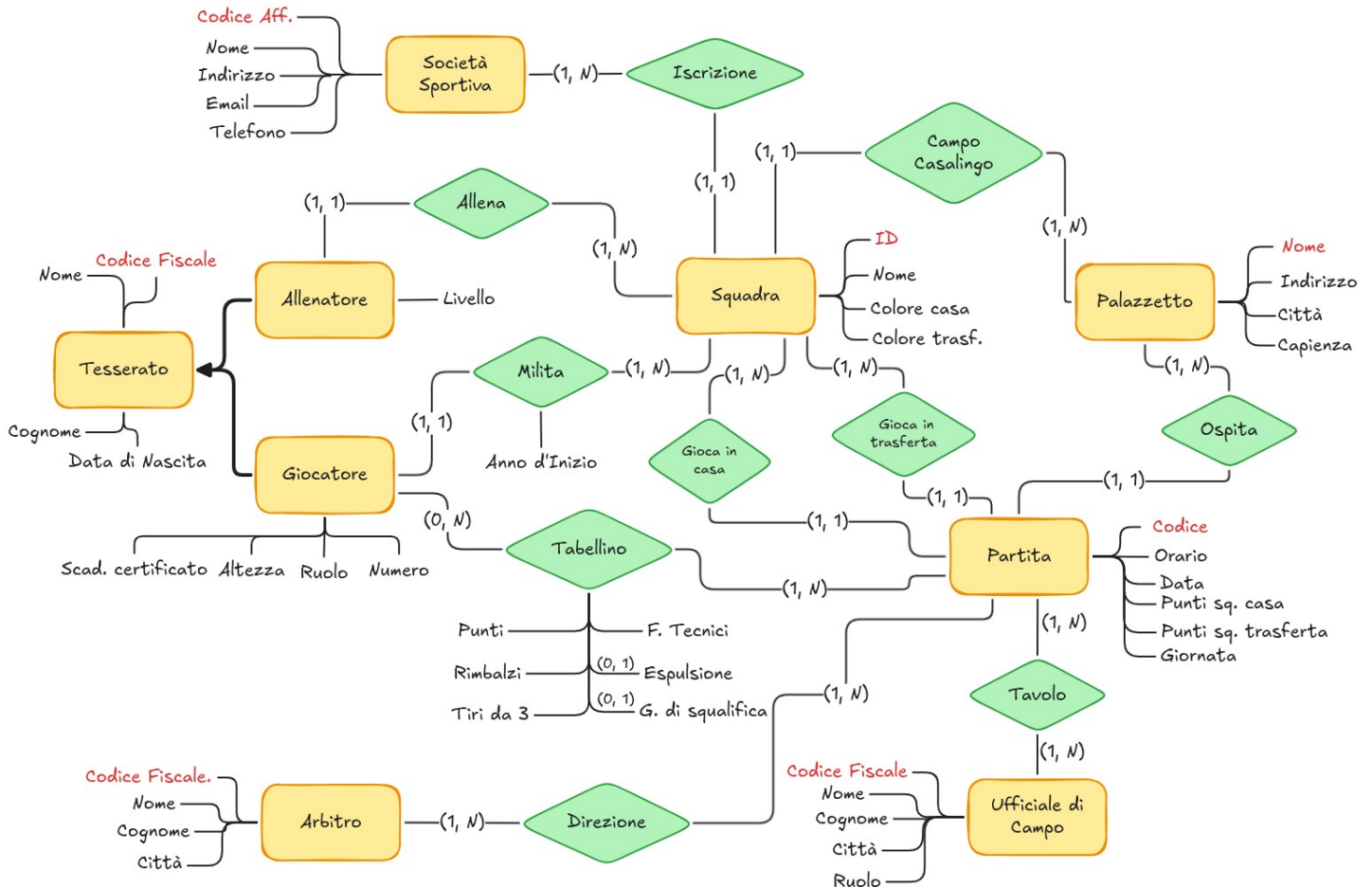
Il campionato è strutturato in partite, in cui si affrontano due squadre distinte, una di casa e una in trasferta. Le partite sono identificate da un codice, la data in cui si svolgono, l'orario d'inizio e la giornata del campionato a cui appartengono.

Per ogni partita sono designati due arbitri, di cui si vuole conoscere nome, cognome, Codice Fiscale e città di provenienza; oltre agli arbitri nelle gare sono presenti anche fino a tre ufficiali di campo (ad es. refertista, cronometrista e addetto ai 24 secondi), identificati da nome, cognome, Codice Fiscale, città di provenienza e ruolo.

Per ogni partita disputata si tiene conto del risultato e delle prestazioni dei singoli giocatori tramite il tabellino, che conterrà punti realizzati, rimbalzi presi e tiri da tre punti

segnati; si vuole inoltre ricordarsi delle sanzioni disciplinari (falli tecnici commessi, l'espulsione (se avvenuta) e le eventuali giornate di squalifica assegnate).

Schema Entity-Relationship:



Dizionario dei dati (entità):

Entità	Descrizione	Attributi	Identificatore
Società Sportiva	Ente che possiede le squadre e le gestisce	Codice affiliazione, nome, indirizzo della sede, e-mail, telefono	Codice affiliazione
Squadra	Gruppo di atleti iscritto al campionato	ID, nome commerciale, colore casa, colore trasferta	ID
Tesserato	Membro ufficiale della squadra, allenatore o giocatore	Codice Fiscale, nome, cognome, data di nascita	Codice Fiscale
Giocatore	Membro della squadra che scende in campo nelle partite	Codice Fiscale, nome, cognome, data di nascita, altezza, ruolo, numero di maglia, scadenza certificato	Codice Fiscale

Allenatore	Membro della squadra che non scende in campo	Codice Fiscale, nome, cognome, data di nascita, livello tesserino	Codice Fiscale
Partita	Singolo incontro sportivo tra due squadre	Codice Partita, orario, data, punti squadra di Casa, punti squadra in trasferta, giornata	Codice Partita
Palazzetto	Struttura dove si svolgono le partite	Nome, indirizzo, città, capienza	Nome
Arbitro	Ufficiale di gara che dirige la partita	Codice Fiscale, nome, cognome, città di provenienza	Codice Fiscale
Ufficiale di Campo	Altri ufficiali addetti a ruoli specifici (cronometro, punti, 24 secondi)	Codice Fiscale, nome, cognome, città di provenienza, ruolo	Codice Fiscale

Dizionario dei dati (relazioni):

Relazione	Descrizione	Componenti	Attributi
Iscrizione	Iscrizione di squadre al campionato	Società Sportiva, Squadra	
Allena	Collega un allenatore alla squadra che dirige	Allenatore, Squadra	
Milita	Collega un giocatore alla sua squadra attuale	Giocatore, Squadra	Anno d'Inizio
Campo Casalingo	Collegamento tra una squadra e il palazzetto in cui gioca le sue gare casalinghe	Squadra, Palazzetto	
Ospita	Palazzetto in cui si svolge una specifica partita	Palazzetto, Partita	
Gioca in casa	Relaziona la squadra alle partite che gioca in casa	Squadra, Partita	
Gioca in trasferta	Relaziona la squadra alle partite che gioca in trasferta	Squadra, Partita	
Tabellino	Collega un giocatore ad una partita in cui ha partecipato, tenendo traccia della prestazione individuale	Giocatore, Partita	Punti, Rimbalzi, Tiri da 3, Falli Tecnici, Espulsione (opzionale), Giornate di squalifica (opzionale)
Direzione	Lega un arbitro ad una partita che dirige	Arbitro, Partita	
Tavolo	Lega un Ufficiale di Campo ad una partita in cui presta servizio	Ufficiale di Campo, Partita	

Considerazioni:

Si noti che la relazione “Iscrizione” che lega Società Sportiva a Squadra è stata creata come “uno a molti” e non “uno a uno” poiché si vuole accettare il caso in cui una stessa società possa iscrivere più squadre nello stesso campionato, cosa totalmente plausibile nelle leghe “minori” come quella considerata in questo progetto (un esempio potrebbe essere una squadra di Under 19 iscritta allo stesso campionato della prima squadra).

Similmente si osservi che la relazione "Campo Casalingo" che collega Squadra a Palazzetto è stata creata anch'essa come "uno a molti" e non "uno a uno" per accettare il caso in cui più squadre (della stessa Società Sportiva e non) possano disputare partite nella stessa struttura.

Si vuole inoltre chiarire la scelta di inserire due relazioni che collegano Squadra a Partita ("Gioca in casa" e "Gioca in trasferta") invece di una unica. Questa scelta è stata fatta per poter memorizzare efficientemente l'informazione su chi sia la squadra di casa e chi quella in trasferta, informazione usata poi per scopi come la stampa dei punteggi o la distinzione tra partite di andata e di ritorno.

Vincoli non esprimibili:

- 1) Gli arbitri e gli Ufficiali di Campo che prestano servizio non possono provenire dalle città dove hanno sede le squadre che si stanno affrontando
- 2) Non possono essere inserite nel calendario due partite diverse che si svolgono nella stessa data, nello stesso palazzetto e in orari coincidenti o sovrapposti (considerando una durata della partita di 2 ore)
- 3) Un giocatore può ricevere giornate di squalifica solo se in quella partita è stato anche espulso
- 4) Un giocatore può prendere parte ad una partita solo se il suo certificato medico è ancora valido, ovvero se la data della partita è antecedente a quella di scadenza del suo certificato

Operazioni di interesse:

Le operazioni che si vogliono svolgere sul database sono:

- 1) Si vuole mostrare il calendario per ogni giornata di partite, comprendendo le squadre che si affrontano, il luogo, la data e l'orario dell'incontro.
(Batch - 1/settimana)
- 2) Si vuole mostrare la classifica del campionato fino ad una certa giornata, indicando i nomi della squadra in ordine di punteggio, calcolato nel seguente modo: vittoria = 2 punti, sconfitta = 0 punti.
(Interattiva - 10/giorno)
- 3) Si vuole assegnare il premio di MVP alla fine del campionato; lo si assegnerà al giocatore che ha fatto più punti e rimbalzi, e che ha ottenuto meno falli tecnici. (Batch - 1/anno)

4) Si vuole ottenere una lista dei giocatori con certificato medico in scadenza entro un mese per inviare un sollecito a rinnovarla. (Batch – 1/mese)

Tabella dei volumi (entità):

Entità	Volume	Note
Squadra	15	Numero di squadre nel campionato
Società Sportiva	13	Supponendo che alcune società iscrivano più squadre
Tesserato	265	Numero di giocatori e allenatori totali
Giocatore	225	Circa 15 per ogni squadra
Allenatore	40	Ad ogni squadra 2 o 3 allenatori
Partita	210	30 giornate di partite (ogni squadra deve giocare contro tutte le altre all'andata e al ritorno), ogni giornata 7 partite (una squadra riposa), $30 \times 7 = 210$
Palazzetto	10	Supponendo che alcune squadre usino lo stesso palazzetto
Arbitro	20	Circa 2 per ognuna delle 7 partite della giornata
Ufficiale di Campo	30	Circa 3 o 4 per ogni partita della giornata

Tabella dei volumi (relazioni):

Relazione	Volume	Note
Iscrizione	15	Ogni squadra del campionato deve essere iscritta
Allena	40	Ogni allenatore fa parte di una squadra (si tiene conto solo di allenatori attivi)
Milita	225	Ogni giocatore fa parte di una squadra (si tiene conto solo di giocatori attivi)
Campo Casalingo	15	Tutte le squadre hanno un palazzetto
Ospita	210	Ogni partita è svolta in un palazzetto
Gioca in casa	210	Ogni partita ha una squadra di casa
Gioca in trasferta	210	Ogni partita ha una squadra in trasferta
Tabellino	5040	In ognuna delle 210 partite sono a referto (quindi hanno

		un tabellino) 12 giocatori per squadra (210*24)
Direzione	420	Circa 2 arbitri per tutte le 210 partite
Tavolo	630	Circa 3 ufficiali di campo per tutte le 210 partite

Ristrutturazione schema Entity – Relationship:

Analisi delle ridondanze:

Prima ridondanza osservata:

Si osserva che gli attributi “Punti squadra di casa” e “Punti squadra in trasferta” dell’entità “partita” sarebbero calcolabili effettuando la somma dei punti segnati dai singoli giocatori a referto (24 in totale), tramite l’attributo “punti” della relazione “tabellino”.

Per valutare se è giustificata la presenza di questa ridondanza si analizzerà il numero di accessi richiesti per l’operazione che necessita il calcolo dei punteggi per determinare il numero di vittorie: *“Si vuole mostrare la classifica del campionato fino ad una certa giornata, indicando i nomi della squadra in ordine di punteggio, calcolato nel seguente modo: vittoria = 2 punti, sconfitta = 0 punti.*

(Interattiva - 10/giorno)”.

In particolare si prende in esame il caso peggiore (più costoso) in cui può venire effettuata questa operazione, ovvero il momento in cui il campionato è concluso e tutte le 210 partite sono state disputate.

- Presenza di ridondanza:

Nel caso vengano mantenuti gli attributi in questione per completare l’operazione occorrerebbe accedere a tutte le 210 partite giocate e fare un semplice confronto tra il punteggio della squadra di casa e in trasferta, assegnando i punti a dovere. Totale accessi richiesti: $210 * 10 = 2100$ accessi al giorno in lettura all’entità “Partita”. (effettuata 10 volte al giorno).

- Assenza di ridondanza:

Nel caso in cui, invece, vengano rimossi gli attributi del punteggio per le squadre di casa e in trasferta, bisognerebbe per ognuna delle 210 partite calcolare il punteggio totale della squadra facendo la somma dei punteggi singoli dei 24 giocatori a referto.

Accessi richiesti:

- $210 \times 24 \times 10 = 50400$ accessi al giorno in lettura alla relazione "Tabellino"
- $210 \times 10 = 2100$ accessi al giorno in lettura all'entità "Partita"

Totale: 52500 accessi al giorno in lettura.

Dato che il numero di accessi richiesti senza ridondanza risulta essere 25 volte maggiore di quello con la ridondanza, e dato che l'operazione verrà effettuata molto di frequente, si decide di tenere questa ridondanza.

Seconda ridondanza osservata:

Si può notare che le tre entità "Squadra", "Partita" e "Palazzetto" formano un ciclo.

In particolare l'analisi verrà fatta per lo svolgimento dell'operazione *"Si vuole mostrare il calendario per ogni giornata di partite, comprendendo le squadre che si affrontano, il luogo, la data e l'orario dell'incontro. (Batch - 1/settimana)"*.

Bisognerà valutare se è giustificato il mantenimento della relazione "ospita" o se si possa rimuovere.

- Presenza di ridondanza:

Per stampare nel calendario di giornata il luogo dove si svolgerà la partita, sarà sufficiente leggerlo come attributo dall'entità "Palazzetto", passando quindi per la relazione "Ospita".

- Assenza di ridondanza:

Nel caso in cui la suddetta relazione venga rimossa, per estrapolare l'indirizzo del palazzetto a partire dall'entità "Partita" occorrerà passare per la relazione "Gioca in casa", arrivando alla squadra di casa, e poi passare per la relazione "Campo casalingo" per arrivare a leggere l'indirizzo.

Se si supponesse che il palazzetto in cui si giocano le partite coincidesse sempre con quello della squadra di casa allora il ciclo sarebbe del tutto ridondante, in quanto passare per la relazione "Ospita" oppure usare "Gioca in casa" e poi "Campo casalingo" darebbe in ogni caso lo stesso risultato.

Tuttavia, non sarebbe del tutto corretto fare questa supposizione, in quanto in campionati minori è abbastanza comune che qualche palazzetto sia indisponibile ad ospitare la partita per svariate motivazioni, obbligando quindi le squadre a giocare in palazzetti alternativi.

Un altro caso in cui questa supposizione non reggerebbe è quello delle "Final 4", spesso e volentieri giocate in un campo neutro, che quindi ancora una volta non coinciderebbe con il campo di nessuna delle due squadre.

Assodato, quindi, che questa supposizione non possa essere fatta, va da sé che la relazione “Ospita” va mantenuta, in quanto così facendo sarebbe contemplato il caso in cui il palazzetto non sia quello della squadra di casa.

Eliminazione delle generalizzazioni:

Lo schema Entity – Relationship costruito presente una sola generalizzazione, quella tra “Tesserato” (padre) e “Giocatore” e “Allenatore” (figli).

Si può notare che la generalizzazione in questione è totale ed esclusiva, ovvero ogni istanza sarà necessariamente identificata come “Giocatore” o “Allenatore”, e non entrambi contemporaneamente.

Si osserva, inoltre, che l’entità padre non è collegata a nessuna relazione, infatti le relazioni che la possono riguardare sono collegate esclusivamente ai figli.

Per questa serie di motivi si sceglie di accorpare l’entità padre ai figli, ovvero di rimuoverla e di associare gli attributi di “Tesserato” (Codice Fiscale, nome, cognome, data di nascita) direttamente ad entrambi i figli.

Partizionamento/accorpamento di entità e relazioni:

Si potrebbe valutare di partizionare l’entità “Giocatore”, che dopo l’eliminazione dell’entità “Tesserato” si ritroverebbe con un numero abbastanza grande di attributi (Codice Fiscale, nome, cognome, data di nascita, altezza, ruolo, numero di maglia, scadenza certificato medico), aggiungendo un’altra entità per dividere gli attributi anagrafici da quelli “sportivi”.

Tuttavia si sceglie di non effettuare questo cambiamento, in quanto nella maggior parte dei casi queste due categorie di attributi verranno visitate entrambe; partizionando, quindi, si andrebbe solamente ad aumentare il numero di accessi in lettura.

Si potrebbe fare lo stesso ragionamento per la relazione “Tabellino”, ovvero quello di dividere gli attributi relativi a statistiche “tecniche” (punti, rimbalzi, tiri da 3 punti) da quelli di natura disciplinare (falli tecnici, espulsioni, giornate di squalifica).

Anche in questo caso si applica lo stesso criterio usato precedentemente, basato sul fatto che anche questi attributi verrebbero acceduti contemporaneamente nella gran parte dei casi.

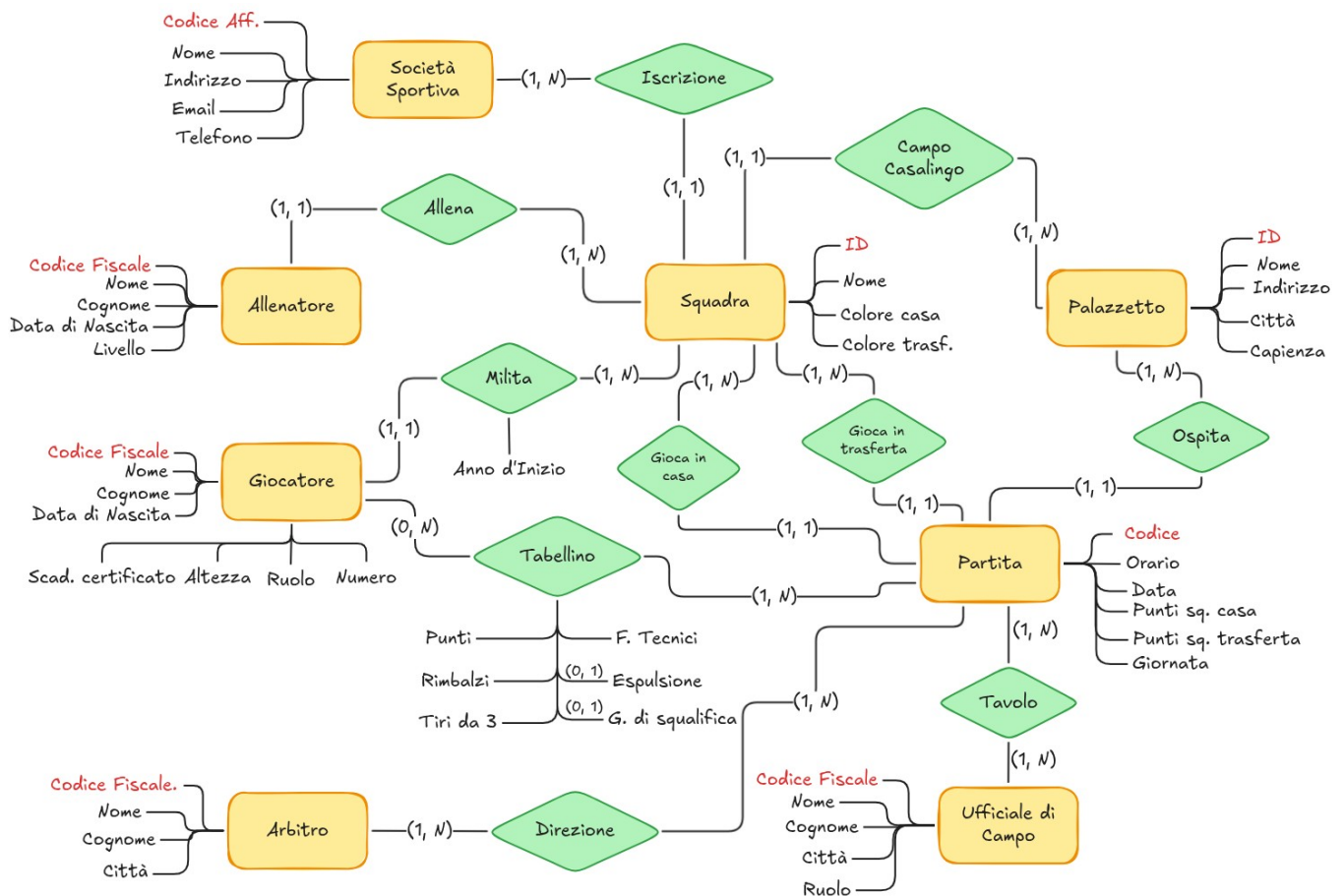
Non sono presenti, invece, attributi multivalore da eliminare.

Scelta degli identificatori primari:

- Società Sportiva: Codice Affiliazione

- Squadra: ID
- Giocatore: Codice Fiscale
- Allenatore: Codice Fiscale
- Arbitro: Codice Fiscale
- Ufficiale di Campo: Codice Fiscale
- Palazzetto: ID (si sceglie di inserire un codice univoco invece del nome per evitare problemi con cambiamenti di nome o palazzetti con nomi generici)
- Partita: Codice Partita

Schema Entity - Relationship ristrutturato:

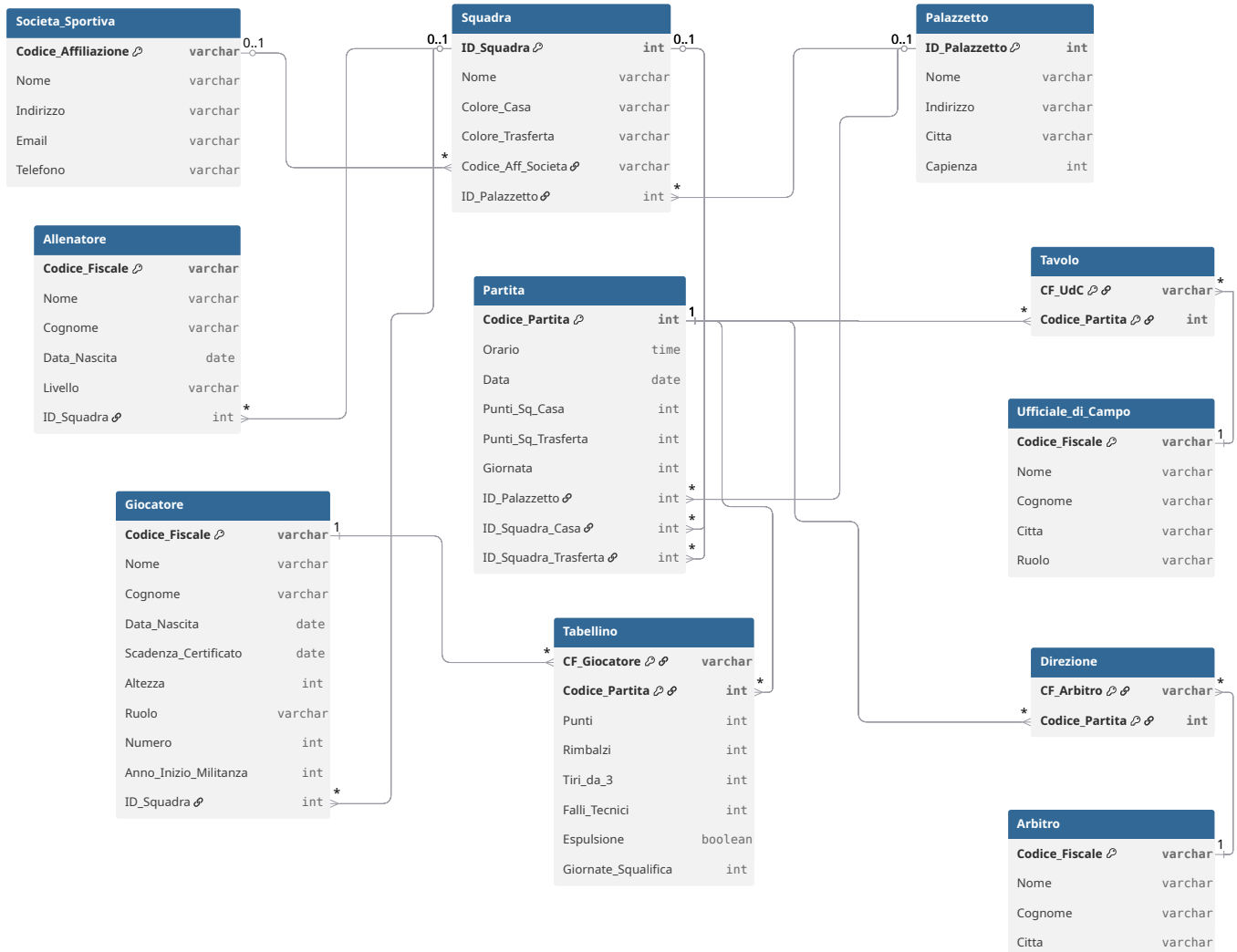


Passaggio allo schema logico:

Verrà usata la sottolineatura per indicare le chiavi primarie, e il *corsivo* per le chiavi esterne.

- Società sportiva (Codice affiliazione, nome, indirizzo, e-mail, telefono)
- Squadra (ID, nome, colore casa, colore trasferta, *codice_aff_società*, *ID_palazzetto*)

- ## Schema logico:



Normalizzazione:

- **Prima forma normale:**

Il database rispetta la prima forma normale in quanto non sono presenti colonne con valori multipli o ripetuti.

- **Seconda forma normale:**

Il database rispetta la seconda forma normale; lo si può vedere dal fatto che le tabelle "Direzione" e "Tavolo" che presentano chiavi primarie multiple non hanno altre colonne, e quindi il problema della dipendenza da entrambi i campi della chiave primaria non si pone.

Per quanto riguarda la tabella "Tabellino", invece, si può notare come tutte le statistiche del giocatore dipendano sia dal suo Codice Fiscale (ci dice che giocatore ha fatto quella determinata performance), sia dal codice della partita (ci dice in che partita quel giocatore ha registrato quelle statistiche).

- **Terza forma normale:**

Si potrebbe notare che nella tabella "Partita" l'attributo "ID_Palazzetto" potrebbe essere ricavato da "ID_Squadra_Casa", ma, come si è già discusso nella sezione di analisi delle ridondanze, potrebbe capitare che il palazzetto non coincida sempre con quello della squadra di casa, quindi non sempre potrebbe essere possibile ricavare il palazzetto della partita passando per la squadra di casa.

Il database rispetta la terza forma normale.

Indici:

Si decide di introdurre un indice sulla colonna "Giornata" della tabella "Partita".

Questa aggiunta è stata pensata per ottimizzare l'esecuzione dell'operazione di stampa del calendario, che richiede di accedere consecutivamente a partite della stessa giornata. Tuttavia, queste potrebbero non essere vicine cronologicamente, ad esempio nei casi di partite annullate e recuperate molto tempo dopo, che però faranno comunque parte della giornata originaria.

Dunque, l'indice sarà di tipo secondario e denso: secondario perché non avrà lo stesso ordinamento della colonna originaria, e denso perché non sarebbe possibile determinare la posizione di un'istanza se non fosse presente nell'indice, per via della mancanza di ordinamento.

```
CREATE INDEX indiceGiornata ON Partita(Giornata)
```

Realizzazione vincoli non esprimibili:

- 1) Gli arbitri e gli Ufficiali di Campo che prestano servizio non possono provenire dalle città dove hanno sede le squadre che si stanno affrontando

```

DELIMITER $$
CREATE TRIGGER ControlloArbitriUdC
BEFORE INSERT ON Direzione
FOR EACH ROW
BEGIN
DECLARE cittaArbitro VARCHAR(100);
DECLARE cittaCasa VARCHAR(100);
DECLARE cittaTrasf VARCHAR(100);
-- Cerco la città dell'arbitro
SELECT Citta INTO cittaArbitro
FROM Arbitro
WHERE NEW.CF_Arbitro = Codice_Fiscale;
-- Cerco la città della squadra di casa (tramite il palazzetto)
SELECT Citta INTO cittaCasa
FROM Partita p
JOIN Squadra s ON p.ID_Squadra_Casa = s.ID_Squadra
JOIN Palazzetto pal USING(ID_Palazzetto)
WHERE NEW.Codice_Partita = p.Codice_Partita;
-- Cerco la città della squadra in trasferta
SELECT Citta INTO cittaTrasf
FROM Partita p
JOIN Squadra s ON p.ID_Squadra_Trasferta = s.ID_Squadra
JOIN Palazzetto pal USING(ID_Palazzetto)
WHERE NEW.Codice_Partita = p.Codice_Partita;
-- Faccio il controllo
IF cittaArbitro = cittaCasa OR cittaArbitro = cittaTrasf THEN
    SIGNAL sqlstate '45001' SET message_text = 'Conflitto di
    interessi!';
END IF;
END $$
DELIMITER ;

```

Si omette il codice del trigger per il controllo sugli ufficiali di campo, essendo identico a quello per gli arbitri, in particolare sostituendo le tabelle "Direzione" e "Arbitro" con "Tavolo" e "Ufficiale di campo"

- 2) Non possono essere inserite nel calendario due partite diverse che si svolgono nella stessa data, nello stesso palazzetto e in orari coincidenti o sovrapposti (considerando una durata della partita di 2 ore)

```

DELIMITER $$
CREATE TRIGGER controlloOrari
BEFORE INSERT ON Partita
FOR EACH ROW
BEGIN
DECLARE contoSovrapposizioni INT DEFAULT 0;
SELECT COUNT(*) INTO contoSovrapposizioni
FROM Partita p
WHERE p.ID_Palazzetto = NEW.ID_Palazzetto AND p.Data = NEW.Data AND

```

```

((NEW.Orario > p.Orario AND NEW.Orario < ADDTIME(p.Orario, '02:00:00')) OR
(ADDTIME(NEW.Orario, '02:00:00') > p.Orario AND
ADDTIME(NEW.Orario, '02:00:00') < ADDTIME(p.Orario, '02:00:00')));
/*Sto controllando se l'orario d'inizio della nuova partita
(NEW.Orario) è compreso tra l'inizio e la fine di una
partita già esistente, oppure se la fine della nuova
partita (ADDTIME(NEW.Orario, '02:00:00')) è compreso tra
l'inizio e la fine di una partita già esistente*/
IF contoSovrapposizioni > 0 THEN
    SIGNAL sqlstate '45002' SET message_text = 'Partite sovrapposte!';
END IF;
END $$
DELIMITER ;

```

- 3) Un giocatore può ricevere giornate di squalifica solo se in quella partita è stato anche espulso

```

DELIMITER $$
CREATE TRIGGER controlloGiornatediSqualifica
BEFORE INSERT ON Tabellino
FOR EACH ROW
BEGIN
    IF NEW.Giornate_Squalifica > 0 AND NEW.Espulsione = FALSE THEN
        SIGNAL sqlstate '45003' SET message_text = 'Errore sanzioni!';
    END IF;
END $$
DELIMITER ;

```

- 4) Un giocatore può prendere parte ad una partita solo se il suo certificato medico è ancora valido, ovvero se la data della partita è antecedente a quella di scadenza del suo certificato

```

DELIMITER $$
CREATE TRIGGER controlloCertificato
BEFORE INSERT ON Tabellino
FOR EACH ROW
BEGIN
    DECLARE scadCert DATE;
    DECLARE dataPartita DATE;
    -- Ottengo la data di scadenza del certificato
    SELECT Scadenza_Certificato INTO scadCert
    FROM Giocatore
    WHERE Codice_Fiscale = NEW.CF_Giocatore;
    -- Ottengo la data della partita
    SELECT Data INTO dataPartita
    FROM Partita
    WHERE Codice_Partita = NEW.Codice_Partita;
    -- Faccio il controllo
    IF scadCert < dataPartita THEN
        SIGNAL sqlstate '45004' SET message_text = 'Certificato scaduto!';
    END IF;
END $$
DELIMITER ;

```

Realizzazione delle operazioni:

1) Si vuole mostrare il calendario per ogni giornata di partite, comprendendo le squadre che si affrontano, il luogo, la data e l'orario dell'incontro.

(Batch - 1/settimana)

```
DELIMITER $$
CREATE PROCEDURE stampaCalendarioGiornata(IN giorn INT)
BEGIN
    SELECT sc.Nome AS SquadraDiCasa, st.Nome AS SquadraInTrasferta, pal.Indirizzo,
    p.Data, p.Orario
    FROM Partita p
    JOIN Palazzetto pal USING(ID_Palazzetto)
    JOIN Squadra sc ON p.ID_Squadra_Casa = sc.ID_Squadra
    JOIN Squadra st ON p.ID_Squadra_Trasferta = st.ID_Squadra
    WHERE p.giornata = giorn;
END $$
DELIMITER ;
```

2) Si vuole mostrare la classifica del campionato fino ad una certa giornata, indicando i nomi delle squadre in ordine di punteggio, calcolato nel seguente modo: vittoria = 2 punti, sconfitta = 0 punti.

(Interattiva - 10/giorno)

(Creo prima una funzione d'appoggio)

```
DELIMITER $$
CREATE FUNCTION calcolaPunti(puntiSq1 INT, puntiSq2 INT) RETURNS INT DETERMINISTIC
BEGIN
    IF puntiSq1 > puntiSq2 THEN
        return 2;
    ELSE
        RETURN 0;
    END IF;
END $$
DELIMITER ;
```

```
DELIMITER $$
CREATE PROCEDURE mostraClassifica(IN giornataLimite INT)
BEGIN
    -- Conto prima quante ne hanno vinte in casa e poi in trasferta
    SELECT nomeSquadra, SUM(puntiPartita) as PuntiTotali
    FROM (SELECT s.nome AS nomeSquadra, calcolaPunti(p.Punti_Sq_Casa,
    p.Punti_Sq_Trasferta) AS puntiPartita
    FROM Partita p
    JOIN Squadra s ON p.ID_Squadra_Casa = s.ID_Squadra
    WHERE p.Giornata <= giornataLimite
    UNION ALL
    SELECT s.nome AS nomeSquadra, calcolaPunti(p.Punti_Sq_Trasferta,
    p.Punti_Sq_Casa) AS puntiPartita
    FROM Partita p
    JOIN Squadra s ON p.ID_Squadra_Trasferta = s.ID_Squadra
```

```

        WHERE p.Giornata <= giornataLimite) punteggiSingoli
    GROUP BY nomeSquadra
    ORDER BY PuntiTotali DESC;
END $$
DELIMITER ;

```

Spiegazione:

La subquery crea una tabella (chiamata punteggiSingoli) con, per ogni partita in casa e in trasferta di ogni squadra, due colonne: il nome della squadra e se in quella singola partita ha preso 2 o 0 punti, attraverso la funzione calcolaPunti. In particolare, questo compito viene svolto concatenando verticalmente due query, la prima che scorre le partite e dice quanti punti hanno fatto le squadra in casa, la seconda che scorre le partite e dice quanti punti hanno fatto le squadra in trasferta.

Successivamente, questa tabella creata tramite subquery viene utilizzata per raggruppare per il nome della squadra e fare la somma dei punti ottenuti nelle singole partite; infine, ordinando si ottiene la classifica.

3) Si vuole assegnare il premio di MVP alla fine del campionato; lo si assegnerà al giocatore che ha fatto più punti e rimbalzi, e che ha ottenuto meno falli tecnici. (Batch - 1/anno)

```

DELIMITER $$
CREATE PROCEDURE nominaMVP()
BEGIN
    SELECT g.Nome, g.Cognome, (SUM(t.Punti)+SUM(t.Rimbalzi)-SUM(t.Falli_Tecnici))
as MVPscore
    FROM Tabellino t
    JOIN Giocatore g ON t.CF_Giocatore = g.Codice_Fiscale
    GROUP BY t.CF_Giocatore, g.Nome, g.Cognome
    ORDER BY MVPscore DESC
    LIMIT 1;
END $$
DELIMITER ;

```

4) Si vuole ottenere una lista dei giocatori con certificato medico in scadenza entro un mese per inviare un sollecito a rinnovarla. (Batch - 1/mese)

```

DELIMITER $$
CREATE PROCEDURE mostraCertificatiinScadenza()
BEGIN
    SELECT g.Nome, g.Cognome
    FROM Giocatore g
    WHERE g.Scadenza_Certificato BETWEEN CURDATE() AND
        DATE_ADD(CURDATE(), INTERVAL 1 MONTH);
END $$
DELIMITER ;

```